CODE REVIEW Ã· 27 AUGUST 2026

AlchemyCloud Audit

LiDAR PCD Editor & Occupancy Grid Exporter Ã· v1.2.0

A full review of the application: every subsystem read, every claim adversarially verified, and the consequential ones reproduced against the repository's own 2.2 M-point scan. The compute core is excellent. The damage is concentrated in three places â€” RANSAC plane selection, the save path, and the release pipeline.

3,182	830	44/44	57	12
LOC APP	LOC TESTS	TESTS PASS	FINDINGS	HIGH SEVERITY
RED				
CI STATUS				

Remediation status

Audited at `9a8eb16` (v1.2.0); remediated by `a953745..cbcc1c4` (v1.3.0). Every finding below carries its current state, so this stays a live record rather than a snapshot of a tree that no longer exists. The OPEN and PARTIAL entries are the backlog worth acting on.

Counts cover the 50 itemised findings *plus* the architecture recommendations the analysis lenses raised â€” which add two Open (full-cloud undo snapshots; the selection algebra still trapped in the GUI) and three Won't fix (the `Tool` strategy, a `Document` model, command-object undo). The chips in the tables below therefore total 39 / 5 / 5 / 1.

39	5	7	4
FIXED	PARTIAL	OPEN	WON'T FIX

The bottom line

VERDICT

This is well-built software with a few sharp edges that happen to sit on the critical path. The separation between the Qt shell and the pure-compute core is real, not aspirational â€” I ran the full `load â†’ voxel â†’ SOR â†’ level_ground â†’ build â†’ save` pipeline with PyQt never entering `sys.modules`. The threading model is careful and correctly reasoned. The exported map geometry is *correct*, which is the single property that matters most, and I verified it by round-tripping an asymmetric obstacle back out of the PGM.

What lets it down is not architecture but **unguarded assumptions about real data**. Three of the four most serious defects share one root cause: the code trusts whatever RANSAC, Open3D, or the user hands it, and never asks whether the answer is plausible. On your own `ppt_15july.pcd`, "Level Ground" grabs a wall and stands the room on its side; the viewer draws 1,580 of 2.2 million points; and a failed save reports success.

None of it is deep. **The five P0 items below are roughly a day of work**, and they convert this from a tool that needs a knowledgeable operator into one you could hand to a teammate.

Fix before the next release

5 ITEMS Â· P0

"Level Ground" and "Remove Ground" fit *any* dominant plane â€” on your own scan they pick a wall

`alchemycloud/pcd_ops.py:140 Â· :67`

HIGH

REPRODUCED

FIXED

WHAT

`segment_plane` returns the plane with the most inliers, and both function align it to +Z unconditionally. There is no check that the normal is anywhere near vertical â€” the only guard is a *lower* bound at

`pcd_ops.py:160` that skips rotation when the tilt is already tiny. In an indo scan a long flat wall routinely out-votes the floor.

EVIDENCE

```
ppt_15july.pcd ā€” 2,217,416 pts, true Z extent 1.80 m

run0: normal=[0.990 0.109 0.084]  tilt=85.17Ā°  ā†’ Z extent after = 21.55
run1: normal=[0.993 0.114 0.029]  tilt=88.37Ā°  ā†’ Z extent after = 21.66
run2: normal=[0.988 0.112 0.109]  tilt=83.75Ā°  ā†’ Z extent after = 21.53
```

A ground normal must have `|n_z| ā‰ˆ 1`; these are ā‰ˆ 0.08. The wall genuinely wins on inliers ā€” 383,100 points against the floor's 209,849 ā so this is not RANSAC bad luck, it is the expected outcome on this cloud. "Remove Ground" has the identical flaw and deletes a 325k-point wall wh keeping the floor.

WHY IT BITES

The rotated cloud is what becomes the occupancy grid. An 85Ā° rotation turns a floor plan into a vertical cross-section. The status bar does report "Levelled 85.17Ā°" and the result is visible in the viewport, so it is not sil ā€” but it is presented as success, and everything downstream (Z-slab, Z clip, marker origin) is now wrong.

FIX

Constrain the fit. Either restrict RANSAC's input to the lowest ~20 % of Z, loop `segment_plane` until `|n_z| ā‰¥ cos(30Ā°)`. Add a `max_tilt_deg` parameter that *raises* rather than applying an implausible rotation, and have `action_level_ground` ask for confirmation: "*the dominant plane is 85Ā° from horizontal ā€” that looks like a wall, not the floor.*"

A failed save reports "Saved" ā€” Open3D's return value is discarded

HIGH REPRODUCED FIXED

`alchemycloud/pcd_ops.py:18 Ā· main_window.py:565, 589`

The viewer draws 1,580 of 2.2 million points ā€” 0.32 % of its own budget

HIGH REPRODUCED FIXED

`alchemycloud/pcd_ops.py:193`

One NaN or one glitch return breaks the whole pipeline

HIGH REPRODUCED FIXED

`alchemycloud/occupancy.py:82, 103 Ã· pcd_ops.py:12, 237`

CI has never been green, and all seven release tags point at one commit

HIGH REPRODUCED PARTIAL

`.github/workflows/ci.yml:20 Ã· publish.yml:9`

What holds up

VERIFIED, NOT ASSUMED

A review that only lists defects misrepresents the codebase. Each of these was something I actively tried to break and could not.

ā“ **The exported map geometry is correct**
I exported an asymmetric L-shaped obstacle and reconstructed world coordinates from `(pixel, origin, resolution)`: recovered `X[1.95, 5.95] Y[0.95, 2.95]` against a truth of `X[2, 6] Y[1, 3]`. No transpose, no mirror ā€” the `flipud`/origin pairing is right, and it writes genuine P5 binary PGM.

ā“ **The ROS threshold constants are the canonical ones**
`UNKNOWN = 205` with `free_thresh = 0.196` gives p = 0.196078, clearing the threshold by 7.8 Ć— 10āˆ»āµ so `map_server` reads it as unknown rather than free. That margin is razor-thin by design and it was chosen correctly.

ā“ **The projection pipeline is sound under a real camera**
World ā†’ screen ā†’ world round-trips to 0.018 m under an oblique perspective view, points behind the camera are correctly culled via `w > 1e-9`, and crop selection is a strict subset of in-front points. The test suite only ever uses an identity MVP, so this was unverified until now.

ā“ **Crop and eraser operate on the full cloud**
The obvious bug here would be computing a mask from the display-LOD subset and applying it to the full cloud. It doesn't ā€” both handlers take `current_pcd` directly with the viewer's MVP, exactly as the README claims.

ā“ The marker transform is exactly right

`p_local = R(ā€™yaw)Ā·(p ā€™ m)` matches an independently constructed `Rz(yaw)¡µ€(p ā€™ m)` to 4.4 Ć— 10ā^»Ā¹ā^¶. No off-by-inverse, correct operation order, and it deep-copies so the source is untouched.

ā“ The concurrency model is genuinely careful

Explicit re-entrancy guard, all actions disabled while busy, `closeEvent` disconnects in-flight worker signals, `setAutoDelete(False)` with a comment explaining precisely why, bounded undo, `blockSignals` around spin-box range updates. The `QAction` signals are bound to `triggered`, which `setChecked()` does not emit ā€” so the recursion this pattern usually causes cannot happen here.

ā“ Corrupt files are caught at the door

Open3D silently returns an empty cloud for garbage input; `load_pcd` converts that into a real `ValueError`. Garbage, zero-byte, wrong-format, and missing files all fail loudly and correctly.

ā“ Japanese file paths work end to end

A plausible worry given the data directories, and unfounded: `ē,¹ē¾¤ć‰‡ć‰¼ć‚æ.pcd`, `3é˙_å°å›³.ply`, and `å°å›³_å‡¦å›.pgm` all save, reload, and export correctly.

ā“ The compute core is Qt-free and NumPy-2 clean

The full pipeline runs headless with PyQt never imported ā€” batch use is real. And nothing in the package touches any of the APIs NumPy 2.0 removed, despite the environment running NumPy 2.2.6.

Everything else

52 FURTHER FINDINGS

SEV	FINDING	LOCATION	STATUS
HI	Preview dialog writes the *stale* grid when the pre-save rebuild fails, then reports "Saved"	`occupancy_preview.py:292`	FIX
HI	Opening a second file leaves the first file's redo stack live ā€” Ctrl+Y swaps in the old cloud under the new file's name	`main_window.py:544`	FIX
HI	`voxel3d` rebuilds cell indices from float voxel centres and punches one-cell holes	`occupancy.py:188`	FIX

SEV	FINDING	LOCATION	STATUS
	through solid walls		
HI	The uncommitted combo flips the *binary* default so every non-obstacle cell exports as UNKNOWN ā€” zero FREE cells	occupancy_preview.py:203	FIX
HI	`min_points_per_cell` blanks voxel3d maps and is ignored entirely by the tristate policy	occupancy.py:113	FIX
HI	Marker X/Y spins clamp to the cloud bbox, so editing yaw teleports a marker placed outside it	main_window.py:394	FIX
HI	One corrupt token in an ASCII PCD is coerced to 0.0 ā€” map origin moves 100 m, PGM grows 10Ć—, no warning	pcd_ops.py:12	PART
HI	Published to PyPI as MIT while requiring GPLv3 PyQt5 ā€” the shipped app is effectively GPLv3	pyproject.toml:10	OPEN
HI	The suite reports "44 passed" while every GL paint throws and pyqtgraph swallows it ā€” 96 GLErrors per run	tests/conftest.py:4	PART
MD	`apply_prepared_cloud` unconditionally calls `reset_marker()`, destroying the origin pose after *every* edit	embedded_viewer.py:349	FIX
MD	Non-modal preview dialog holds a snapshot of `current_pcd` while the main window stays editable ā€” exports the stale cloud	main_window.py:1068	FIX
MD	Busy state disables only toolbar actions; every menu action and shortcut stays live, and `action_open` has no busy guard	main_window.py:522	FIX
MD	`_on_save_when_ready` keeps polling after the dialog closes, then pops a modal Save dialog	occupancy_preview.py:286	FIX
MD	Z-Slab with low > high: the viewer swaps the bounds but crop/eraser read the raw spins and become silent no-ops	main_window.py:854	FIX
MD	Eraser rasterises isolated discs at sampled mouse points with no interpolation ā€” a fast drag erases a dotted line	embedded_viewer.py:793	FIX
MD	`screen_to_world_xy` accepts negative ray parameters, so clicks above the horizon land behind the camera	embedded_viewer.py:690	FIX

SEV	FINDING	LOCATION	STATUS
MD	Tristate + voxel3d quantises Z to voxel centres, so obstacles just above the threshold can vanish	occupancy.py:183	NO
MD	`padding_cells=0` on a zero-extent axis crashes with `IndexError: index -1 is out of boundsā€¦ size 0`	occupancy.py:105	FIX
MD	`occupancy.save`'s `with_suffix` truncates any filename containing a dot, and clobbers an existing pair with no prompt	occupancy.py:203	FIX
MD	A failed map export tells the user nothing ā€” the exception escapes the Save slot and stale stats stay on screen	occupancy_preview.py:302	FIX
MD	Pure-view operations (Reset camera, LOD toggle) silently destroy the marker pose	embedded_viewer.py:349	FIX
MD	`level_ground` silently ignores `translate_to_zero` when the cloud is already level	pcd_ops.py:160	FIX
MD	The destructive edit paths (crop, eraser, slab scope) have zero coverage ā€” three simultaneous logic inversions still pass 44/44	main_window.py:847	PART
MD	Map geometry is never asserted: hard-coding the YAML origin to (0,0,0) or deleting both flips still passes 44/44	tests/test_occupancy.py:144	FIX
MD	The preview legend swatches are inverted relative to the PGM it renders	occupancy_preview.py:166	FIX
MD	`closeEvent`'s dropped Worker reference does segfault ā€” precondition is a saturated pool, not a running worker	main_window.py:517	FIX
MD	Zero accessible names in the UI, no label buddies, and the file-path label fails WCAG AA at 3.08:1	main_window.py:352	OPEN
MD	`ToggleSwitch` renders pixel-identical when disabled and draws no focus indicator	widgets.py:68	OPEN
MD	`app.py` never opts into Qt5 HiDPI scaling, sets no application identity, and treats any `argv[1]` as a filename	app.py:11	FIX
MD	Exported PGM+YAML carries no provenance, and Japanese map names are written as `\uXXXX` escapes	occupancy.py:208	PART

SEV	FINDING	LOCATION	STATUS
LO	Crop and Marker clear the viewer's eraser state but leave the Eraser toolbar button checked ā€” it takes two clicks to re-enable	main_window.py:649	FIX
LO	Eraser mode swallows every mouse button, so the camera cannot be orbited while erasing (crop mode allows it)	embedded_viewer.py:754	FIX
LO	Every "Generate Map" click leaks a new visible dialog; stale ones stay on screen with live Save buttons	main_window.py:1074	FIX
LO	`crop_with_box` is unreachable dead code whose two branches disagree on inverted bounds	pcd_ops.py:77	OPEN
LO	`remove_ground_ransac` accepts `distance_threshold=0` and returns plane `[0,0,0,0]` with nothing removed	pcd_ops.py:60	FIX
LO	`_ParamDialog` accepts nonsense values; failures surface as a raw Python traceback	main_window.py:42	FIX
LO	No `sys.excepthook` ā€” surviving an escaped slot exception is an accident of a pyqtgraph import side effect	app.py:11	FIX
LO	`negate` is written into the YAML but never applied to pixels, inverting the map if it is ever set	occupancy.py:212	FIX
LO	`level_ground`'s normal-flip branch is 0 % covered; an upside-down cloud reports a small tilt and stays inverted	pcd_ops.py:153	OPEN
LO	The new tristate test doesn't pin the default, and the binary + UNKNOWN combination is untested	tests/test_occupancy.py:20	FIX
LO	`requires-python = ">=3.10"` has no upper bound, but open3d ships no wheel for CPython 3.13+	pyproject.toml:11	FIX
LO	109 MB of untracked-but-unignored material in the repo root, including a 45 MB unrelated PDF and two scripts from a different robot project	.gitignore:42	FIX
LO	No `__version__` anywhere; "1.2.0" is written independently in four places	alchemycloud/__init__.py	FIX
LO	No logging, no log file, no version string in the UI ā€” a user bug report has nothing	app.py	FIX

SEV	FINDING	LOCATION	STATUS
	to attach		
LO	README claims "43 tests"; there are now 44. `version_data.md` dates 1.2.0 to 2026-05, CHANGELOG to 2026-06-02	README.md:369	FIX

Does it fit a robotics workflow?

DOMAIN READ

The file format is right; the *height semantics and the defaults* are what would burn a robotics user. The writer emits a valid binary PGM with `image` as a bare basename (so it resolves relative to the YAML), an `origin` that correctly describes the lower-left cell including padding, and intensities that decode exactly as intended under both servers' own maths. The missing `mode` key is harmless ā€” nav2 falls back to `Trinary` and ROS 1 never reads it. It only starts to matter the day you want nav2 keepout filter masks, which need `mode: scale`.

The default export is a ceiling map

The dialog defaults `z_min` / `z_max` to the cloud's full Z extent and the threshold to 30 % up that range, with tristate on. On your own `TEMP/current.pcd`, 600,082 of 2,647,270 points ā€” 23 % ā€ sit within 5 cm of z = 2.45 m. That is the ceiling, and it is denser than the floor's modal bin. Every dominant-plane heuristic in the app latches onto it, and the default slab includes it, so the first map a new user exports is a picture of the ceiling.

Tristate marks low obstacles as free

A cell is FREE if it has in-slab points and OCC only where `z > threshold`. A 0.30 m pallet under a 0.60 m threshold exports as 254 ā€ drivable. That is the documented behaviour, but the docstring's framing ("threshold ā‰ robot height") invites exactly this mistake. What a nav stack actually wants is an obstacle *band* relative to the detected floor, not a single global cut.

The uncommitted change makes the binary default unsafe

The new combo is genuinely useful for tristate. But wiring one boolean into both policies flips the binary default from FREE to UNKNOWN, and in binary mode "empty"

means *every cell that isn't an obstacle* ā€” so the exported map has literally zero free cells. I confirmed it: binary + "Unknown (grey)" yields `OCC 1600 Ā· FREE 0 Ā· UNK 900`. Keep the combo bound to `tristate_empty_unknown` only, or make its default policy-dependent and relabel it in binary mode.

What the prototypes had and the app lost

Both `occupancy_grid_tuner.py` and `ceiling_tuner.py` shipped argparse CLIs and a 4 M-cell guard. The app has neither. A headless `alchemycloud-map` entry point over the existing `GridParams` would cost very little and would let maps be regenerated from a deploy pipeline rather than by hand.

Architecture

STRUCTURAL READ

The core/shell split is the best decision in the codebase and it genuinely holds. `pcd_ops.py` and `occupancy.py` import only numpy, open3d, scipy, matplotlib, yaml and PIL; nothing reaches back into the view. Both are 100 % return-annotated and fully covered.

About 80 lines of core geometry did not make it across that line, though. The eraser stroke rasterisation and the entire Z-slab scope algebra live inside `MainWindow._on_eraser_stroke` and `_on_crop_requested`, reading `QAction.isChecked()` and `QDoubleSpinBox.value()` directly. That is precisely why those paths have zero test coverage ā€” they cannot be unit-tested without a window.

`main_window.py` at 1,074 lines is not a god object by *size*; it is one by *authority*. It is the only place the app's invariants exist, and they are encoded as coincidence rather than structure. The clearest symptom: the Z-slab bounds have two homes ā€” the viewer normalises `low > high` by swapping, while crop and eraser read the raw spin boxes ā€” so inverted bounds produce a visible slab that cannot be edited.

Undo stores full cloud snapshots. On `ppt_15july.pcd` that measures ~101 MB per `deepcopy`, or ~1 GB at the 10-deep cap. Six of the nine mutating operations are pure subsets, so index-mask command objects would cut that by orders of magnitude.

The `_apply_async` FLAT-tuple protocol is guarded only by prose, and three distinct silent failure modes were confirmed against it. An explicit `OpResult` dataclass would make the contract checkable.

Suggested order of work

ROADMAP

Now

~1 DAY

01 **Constrain the plane fits.**

Restr `level_ground` RANS to the lowe 20 % of Z in both and `remove_ground_ransac`, and refuse a fit whose normal is more than 30Ā° from vertical.

02 **Make save honest.**

Check Open3D's

return value, coerce the extension, add dirty tracking and a close prompt.

03 **Filter non-finite points at load**

and add a cell-count cap before the grid allocates.

04 **Fix the LOD budget**

ā€” swap the cube-root heuristic for uniform random sampling.

05 **Get CI green**

and gate `publish.yml`

on tests plus a tag/version check.

Next

~3 DAYS

01 **Restore the binary FREE default**

before the tristate combo change is committed, and update the CHANGELOG to cover both halves.

02 **Clear both undo stacks on load**

ā€” a one-line fix for a cross-file data-corruption bug.

03 **Make `reset_marker()` conditional**

so ordinary edits stop destroying the origin pose.

04 **Single-source the Z-slab bounds**

throu`viewer.slab_bounds()`

.

05 **Give the preview dialog a live cloud**

ā€” eithe make it moda or conn it to a `cloudChanged`

signal.

06 **Add golden-file tests**

for the PGM/YAML output so an origin or flip regression cannot pass.

07 **Settle the licence**

: either relabel as GPLv3 with a NOTICE, or port to PySide6 (~14 call sites, since pyqtgraph already supports it).

Later

STRUCTURAL

01 **Extract a `Tool` strategy**

to repla the four paral boole toggl and a

```
Document
```

object to own cloud + path + history.

02 **Move the selection algebra into a Qt-free module**

so the destructive paths become testable.

03 **Switch undo to command objects**

for the six operations that are pure subsets.

04 **Add logging and a version string**

so bug reports carry something actionable.

05 **Ship a headless `alchemycloud-map` CLI**

and an obstacle-band export mode referenced to a detected floor.

How this was produced

METHOD

Eighteen agents, then verification, then reproduction

Seven reviewers worked in parallel over separate dimensions ā€” geometry, occupancy, viewer, concurrency, state, robustness, hygiene. Each dimension's findings then went to an independent skeptic instructed to *refute* them, running code wherever a claim could be settled empirically. Three analysis lenses (architecture, robotics fit, maintainability) and a completeness critic ran alongside. Fifty raw findings entered verification; 49 survived and one was refuted.

That low refutation rate deserves scepticism, so I reproduced the consequential claims myself against the repository's own data rather than relaying them. Findings marked REPRODUCED are ones I ran directly. That pass also corrected two of my own earlier conclusions: I had suspected the crop/eraser mutex was functionally broken (the viewer enforces it internally ā€” only the button state desyncs), and that `display_downsample`'s aliasing was a live bug (its one call site already guards it).

Totals: 18 agents, 0 errors, 651 tool calls, ~2.0 M tokens, 66 minutes ā€” plus an independent verification pass covering map round-tripping, projection accuracy, NaN and outlier handling, non-ASCII paths, corrupt-file behaviour, lint status, and tag history.

AlchemyCloud v1.2.0 Ā· reviewed at 9a8eb16 with 5 files uncommitted
Environment: Python 3.10.6 (pyenv pcd_env) Ā· open3d 0.19.0 Ā· NumPy 2.2.6 Ā· PyQt5 5.15.14 Ā· pyqtgraph 0.14.0
44 tests passing in 1.57 s Ā· `ruff` 0.16.4 run from an isolated venv, not installed into pcd_env